Object Fetching Via Waypoint Navigation

Using ROS2, Nav2, rviz2, OpenCV and ArUco Marker Detection

Team: Titan
Siddhant Gudwani (Roll No: 2024102042
Yogesh Krishna (Roll No: 2024102035)
Karedla Venkata Satya Prakash (Roll No: 2025122006)

Platform:	TurtleBot3 Waffle
Framework:	ROS2 Humble
Simulator:	Gazebo 11 (Ignition-compatible)
Language:	Python 3.10
Date:	March 2026

Contents

1 Introduction and Problem Statement **3**
1.1 Motivation . . . 3
1.2 Problem Statement . . . 3
1.3 Key Constraints . . . 3

2 System Overview **4**
2.1 Hardware and Software Stack . . . 4
2.2 High-Level Architecture . . . 4

3 Environment and Map **4**
3.1 Gazebo World . . . 4
3.2 Map Generation (`generate_map.py`) . . . 4
3.3 Exclusion-Zone Sampling (`pickup_site_node.py`) . . . 7

4 Node Descriptions **8**
4.1 `main_controller` — Mission State Machine . . . 8
4.1.1 Our approach . . . 8
4.1.2 The Finite State Machine (FSM) . . . 9
4.1.3 The 2 Hz Timer — How the FSM Actually Runs . . . 10
4.1.4 Approach Point Computation . . . 12
4.1.5 Multi-Angle Retry . . . 12
4.1.6 RViz Status Overlay . . . 12
4.2 `waypoint_navigator` — Nav2 Action Client . . . 12
4.2.1 Public Interface . . . 13
4.2.2 Execution Flow . . . 13
4.3 `marker_detector` — ArUco Visual Detection . . . 13
4.3.1 ArUco Detection Pipeline . . . 13
4.3.2 Detection Parameter Tuning . . . 13
4.4 `task_scheduler` — Zone Ordering . . . 14
4.4.1 Bonus Implementation: Distance-Based (Nearest First) . . . 14
4.4.2 Priority-Based: Was implemented earlier, C . . . 14
4.5 `pickup_site_node` — Gazebo Zone Spawning . . . 14
4.5.1 Random Position Generation . . . 15
4.5.2 Gazebo Spawning . . . 15

5 Nav2 Stack Integration **16**
5.1 Global Planner — A* (NavfnPlanner) . . . 16
5.2 Local Controller — Regulated Pure Pursuit (RPP) . . . 16
5.2.1 RPP Parameters . . . 16
5.3 Costmaps . . . 17
5.3.1 Local Costmap . . . 17
5.3.2 Global Costmap . . . 18
5.4 Localisation — AMCL . . . 18
5.5 Goal Tolerances . . . 19

6 ROS2 Communication Graph **20**
6.1 Topics . . . 20

6.2 Services . . . 20
6.3 Actions . . . 20
6.4 Node Communication Diagram . . . 21

7 Task Scheduling **21**
7.1 Scheduling Strategies . . . 21

8 ArUco Marker Detection and Object Identity Confirmation **22**
8.1 Zone–Marker Mapping . . . 22
8.2 Detection-Confirmation Protocol . . . 22
8.3 Heading Precision for Detection . . . 23

9 Launch System **23**
9.1 Launch File Structure . . . 23
9.1.1 Execution and Process Management . . . 23
9.2 Starting a Mission . . . 23

10 System Integration and Design Decisions **24**
10.1 Single-Process Multi-Node Architecture . . . 24
10.2 RPP over DWB . . . 24
10.3 2-D vs. 3-D Costmap . . . 24
10.4 Randomised Zone Positions . . . 24

11 Results and Testing **25**
11.1 Observed Behaviour . . . 25
11.2 Failure Modes and Mitigations . . . 25

12 Conclusion **25**

Introduction and Problem Statement

Motivation

Autonomous mobile robots capable of navigating structured indoor environments, identifying objects via visual markers, and performing logistics tasks represent a core challenge in robotics research and industrial automation. The ability to sequence multiple pickup-and-delivery operations — selecting them efficiently, detecting the correct object at each site, and returning to a drop-off zone — encapsulates fundamental problems in planning, perception, and control.

Problem Statement

This project implements a fully autonomous **object fetching** mission on a TurtleBot3 Waffle mobile robot operating in a simulated 10 m × 10 m walled room in Gazebo. The environment contains interior partition walls, static obstacles (barrels, crates, a table), and three *pickup zones* whose positions are **randomised at every simulation launch**.

The robot must:

1. Autonomously navigate to each pickup zone using waypoint-based navigation.
2. Detect the ArUco visual marker at the zone to confirm the correct object identity before pickup.
3. Return the object to a fixed *Home Base* drop-off zone.
4. Repeat for all three zones.
5. Order the visits either by *nearest-first* (minimise total travel distance) or by *priority* (predefined importance order).

All navigation must be fully autonomous — no human teleoperation is permitted at any stage of the mission.

Key Constraints

- Pickup zone positions are unknown at compile time (randomised each run).
- The robot must not collide with any static obstacle or wall.
- Object identity must be *confirmed via camera* before proceeding to drop-off; a timeout fallback is the only permissible alternative.
- All components run within a single Docker container (no external services).

System Overview

Hardware and Software Stack

Component	Detail
Robot model	TurtleBot3 Waffle (differential drive, 0.22 m radius)
Simulator	Gazebo 11, software rendering (llvmpipe), 200 Hz physics
ROS2 distribution	Humble
Navigation stack	Nav2 (AMCL + NavfnPlanner + RPP controller)
Localisation	AMCL (Adaptive Monte Carlo Localisation)
Global planner	NavfnPlanner — A* algorithm
Local controller	Regulated Pure Pursuit (RPP)
Perception	OpenCV 4.5 ArUco detector (DICT_4X4_50)
Language	Python 3.10
Container	Docker (Ubuntu 22.04 base image)

High-Level Architecture

The system is divided into five ROS2 Python nodes, each with the following responsibility:

1. `main_controller` — Mission state machine (the brain).
2. `waypoint_navigator`— Nav2 action client (sends navigation goals).
3. `marker_detector` — Camera processing and ArUco detection.
4. `task_scheduler` — Zone ordering (distance or priority).
5. `pickup_site_node` — Random zone spawning in Gazebo.

The architecture follows a **modular, message-passing** design: nodes communicate exclusively through ROS2 topics, services, and actions. Nav2 is used as a *black box* — the project code never implements path-planning or obstacle avoidance; it only issues goals via the `NavigateToPose` action and waits for a success/failure result.

Environment and Map

Gazebo World

The simulated environment is a 10 m × 10 m room with:

- **Outer walls**: 0.15 m thick at $x = \pm 5$, $y = \pm 5$.
- **Interior partitions**: two horizontal walls at $y = \pm 1.5$ (spanning $x = -5$ to $x = -1$) and a vertical wall at $x = 1.5$ with a doorway gap.
- **Static obstacles**: table, three barrels, two crates, one box.
- **Pickup zones**: three coloured platforms (0.6 m × 0.6 m) with ArUco marker cubes, spawned at random positions each run.
- **Home Base**: fixed at the origin (0, 0).

Map Generation (`generate_map.py`)

The static occupancy grid used by Nav2 is produced entirely by `maps/generate_map.py` — no hand-drawn map is involved. The script instantiates a `MapGenerator` object configured for a 10 m × 10 m room at a resolution of 0.05 m/pixel, yielding a 200 × 200 pixel grid.

The grid is initialised as a flat `bytearray` with every cell set to 254 (free space in the Nav2 convention; 0 = occupied, 205 = unknown).

Coordinate Transform

World coordinates (metres, origin at room centre) are converted to pixel indices by:

$$p_x = \left\lfloor \frac{x_{\text{world}} - x_{\text{origin}}}{0.05} \right\rfloor, \qquad p_y = \left\lfloor \frac{y_{\text{world}} - y_{\text{origin}}}{0.05} \right\rfloor \tag{1}$$

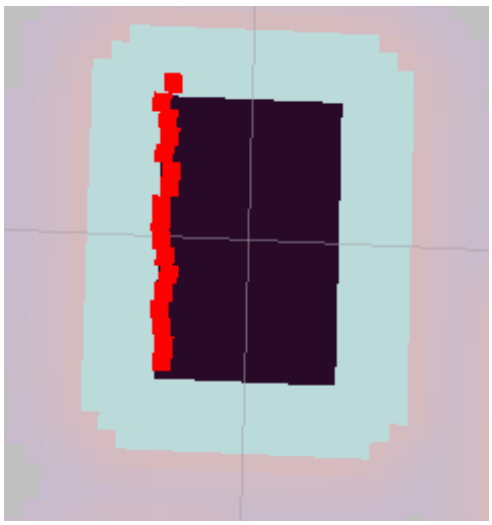
where $x_{\text{origin}} = y_{\text{origin}} = -5.0\,\text{m}$ for our environment. Because the PGM format stores rows top-to-bottom while ROS maps grow bottom-to-top, every write uses a flipped row index:

$$row = \left\lfloor \frac{200 - 1 - p_y}{0.05} \right\rfloor$$

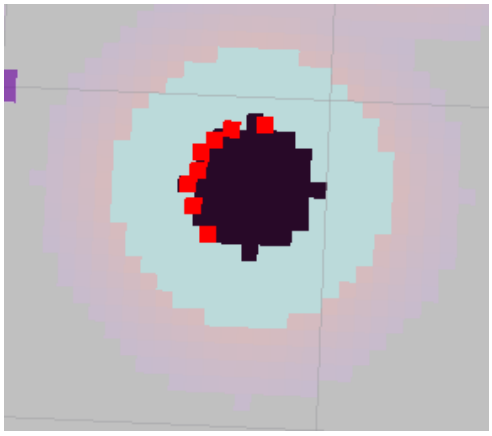
Rasterisation Primitives

Two primitives mark cells as occupied (value 0):

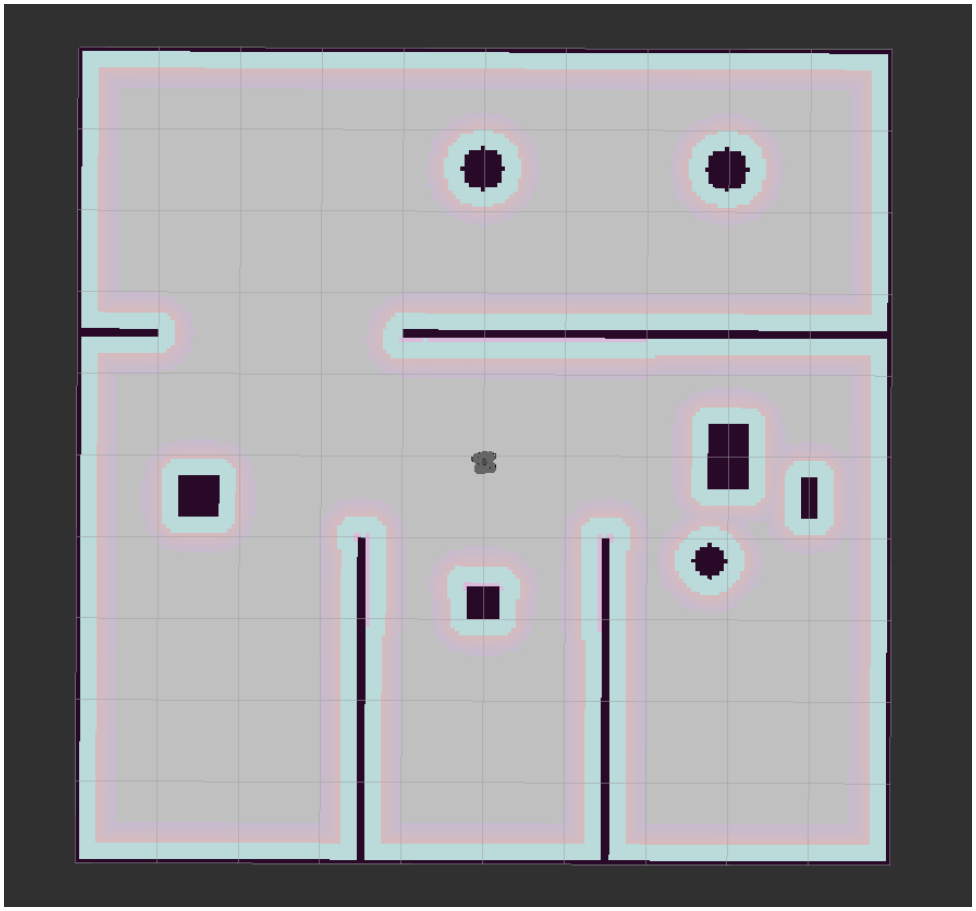
- `add_box(cx, cy, w, h)` — computes the pixel bounding box of an axis-aligned rectangle centred at (cx, cy) with world dimensions $w \times h$, then fills every pixel inside with 0.



- `add_cylinder(cx, cy, radius)` — iterates over the bounding square of the circle and marks pixels satisfying $(c_x - p_x)^2 + (c_y - p_y)^2 \leq r^2$ to reproduce the circular footprint.



Objects Rasterised



The script calls these primitives in the following order:

Call	World position	Size	Object
`add_box`	$(0,\ +5)$	$10.15 \times 0.15\,\mathrm{m}$	North wall
`add_box`	$(0,\ -5)$	$10.15 \times 0.15\,\mathrm{m}$	South wall
`add_box`	$(-5,\ 0)$	$0.15 \times 10\,\mathrm{m}$	West wall
`add_box`	$(+5,\ 0)$	$0.15 \times 10\,\mathrm{m}$	East wall
`add_box`	$(-3,\ -1.5)$	$4 \times 0.15\,\mathrm{m}$	Lower horz. partition
`add_box`	$(-3,\ +1.5)$	$4 \times 0.15\,\mathrm{m}$	Upper horz. partition
`add_box`	$(1.5,\ -2)$	$0.15 \times 6\,\mathrm{m}$	Right vert. partition
`add_box`	$(1.5,\ +4.5)$	$0.15 \times 1\,\mathrm{m}$	Right vert. stub
`add_box`	$(0,\ -3)$	$0.8 \times 0.5\,\mathrm{m}$	Table
`add_cylinder`	$(3.5,\ 0)$	$r = 0.25\,\mathrm{m}$	Barrel 1
`add_box`	$(-0.5,\ 3.5)$	$0.5 \times 0.5\,\mathrm{m}$	Crate 1
`add_cylinder`	$(3.5,\ -3)$	$r = 0.25\,\mathrm{m}$	Barrel 2
`add_box`	$(-1.8,\ 0)$	$0.4 \times 0.4\,\mathrm{m}$	Crate 2
`add_box`	$(-0.5,\ -4)$	$0.6 \times 0.3\,\mathrm{m}$	Box south
`add_cylinder`	$(-1.3,\ -2.8)$	$r = 0.20\,\mathrm{m}$	Barrel 3

Output Files

The finished grid is stored as a binary P5 PGM image (header: `P5 200 200 255`) followed by the raw byte array. This file is paired with a configuration file that provides the necessary context for the robot to map pixels to the coordinate frame:

Listing 1: Map YAML configuration

```
image: map.pgm
resolution: 0.05
origin: [-5.0, -5.0, 0.0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196
```

Nav2's `map_server` loads these files at launch and publishes the occupancy grid on the `/map` topic. The global planner (such as NavFn or A*) then consumes this grid to compute obstacle-free paths, using the `resolution` to convert pixel indices to metric distances and the `origin` to set the map's anchor point in the world frame.

Exclusion-Zone Sampling (`pickup_site_node.py`)

Random zone placement is handled entirely inside `pickup_site_node.py` by the two pure functions `_is_valid()` and `_random_positions()`. No ROS dependency is required for this step; it runs synchronously in `__init__` before any Gazebo calls are made.

Validity Test — `_is_valid(x, y, existing, min_sep=1.5)`

A candidate point (x, y) is accepted only if *all* of the following conditions hold:

1. **Room bounds**: $|x| \leq 4.3$ and $|y| \leq 4.3$ (0.7 m inset from the outer walls).
2. **Exclusion rectangles**: the point does not fall inside any of 16 axis-aligned rectangles encoded as $(c_x,\ c_y,\ h_w,\ h_h)$ tuples. The test is $|x - c_x| < h_w$ **and** $|y - c_y| < h_h$. The rectangles cover outer walls (with margin), interior partitions (with 1.0 m padding each side to keep zones out of narrow corridors), every known static obstacle, and the Home Base at the origin.
3. **Minimum separation**: $\| (x, y) - (e_x, e_y) \|_2 \geq 1.5\,\text{m}$ for every already-placed zone (e_x, e_y), preventing two zones from spawning so close together that the robot cannot manoeuvre between them.

Sampling Loop — `_random_positions(count=3)`

The outer loop runs once per zone (three iterations total). Inside each iteration an inner loop draws uniform random samples:

1. Sample x uniformly in interval $(-4.3,\ 4.3)$, and y uniformly in interval $(-4.3,\ 4.3)$.
2. Pass (x, y) to `_is_valid()`.
3. If valid: append to the accepted list and **break** to the next zone.
4. If invalid: increment attempt counter and repeat.

The actual implementation in `pickup_site_node.py` is:

Listing 2: Double-loop position sampler — `pickup_site_node.py`

```
def _random_positions(count=3):
    fallbacks = [(-2.5, -3.0), (-2.5, 3.0), (3.0, 3.0)]
    positions = []
    for i in range(count):          # outer loop: one iteration per zone
        for _ in range(500):        # inner loop: up to 500 random attempts
            x = random.uniform(-4.3, 4.3)
            y = random.uniform(-4.3, 4.3)
            if _is_valid(x, y, positions):   # exclusion + separation check
                positions.append((x, y))
                break               # success -> move to next zone
        else:
            # Python for-else: runs only if the inner loop
            # exhausted all 500 attempts WITHOUT breaking
            positions.append(fallbacks[i])
    return positions
```

The system tries up to 500 times to find an empty spot for the robot, which almost always works because the room is large. If it fails after 500 tries, it simply uses a pre-set safe position – $\{(-2.5, -3.0), (-2.5, +3.0), (3.0, +3.0)\}$ to prevent the program from getting stuck.

The found valid (x, y) values are rounded to two decimal places and stored in `self._zone_positions` as a dict keyed by `zone_1` / `zone_2` / `zone_3`. They are then passed on to the scheduler's waypoint table by `main_controller._apply_random_zone_positions()` before the first navigation goal is sent.

Node Descriptions

`main_controller` — Mission State Machine

Our approach

`main_controller` controls all the nodes – (scheduler, navigator, detector, spawner), it decides which node in all cases.

The controller does the task of scheduling the activation and deactivation of all the nodes until all three zones are visited.

Sub-node architecture. Rather than launching five separate processes, `main_controller` directly creates the other four nodes inside the same Python process and registers them all with one `MultiThreadedExecutor` (4 threads). This means all five nodes share the same memory this approach reduces latency on service/action calls made internally (e.g. the navigator is called as a regular Python method, not via a ROS service).

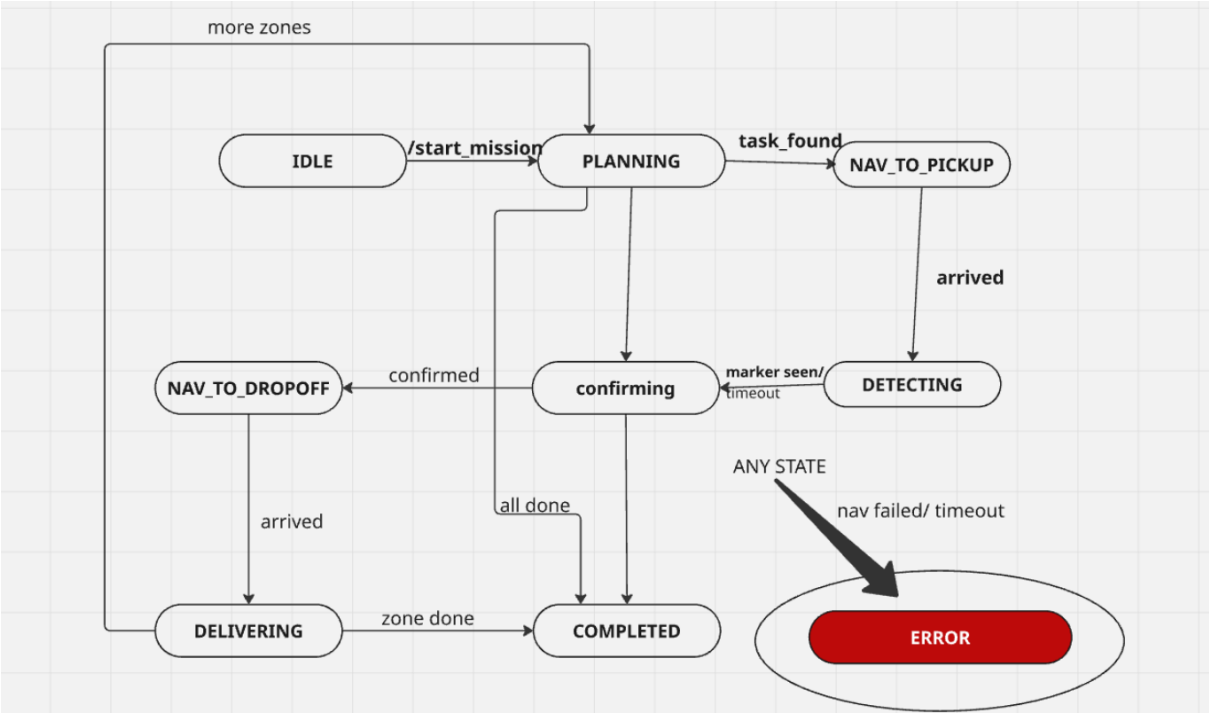
The Finite State Machine (FSM)

A **finite state machine** is used to model the simulation–time states(infinite) into fixed clearly defined states(for implementation), and it can only move to certain other states depending on the stimuli it recieves and the output that the robot provides.

The controller defines **nine states**:

State	What the robot is doing
`IDLE`	Waiting for the `/start_mission` service call. The robot does nothing until explicitly started.
`PLANNING`	Asking the `TaskScheduler` which zone to visit next (nearest unvisited zone, or by priority).
`NAVIGATING_TO_PICKUP`	Driving to the chosen pickup zone. Nav2 plans and executes the path autonomously.
`DETECTING_MARKER`	Arrived at the zone, now watching the camera feed for the expected ArUco marker ID.
`CONFIRMING_PICKUP`	Marker was seen (or timeout elapsed) — logging the pickup confirmation before heading back.
`NAVIGATING_TO_DROPOFF`	Driving to the fixed home-base drop-off point.
`DELIVERING`	Reached home base — recording the delivery, marking the zone complete, 1-second pause.
`COMPLETED`	All three zones done. Mission over.
`ERROR`	A problem occurred (e.g. nav failed on all retries). Call `/start_mission` again to retry.

Figure **??** shows the allowed transitions between states. The arrows point to the next state after the topic labelled above them returns true. The loop back from `DELIVERING` to `PLANNING` is what causes the robot to automatically pick the next zone without any human input.



The 2 Hz Timer — How the FSM Actually Runs

The FSM updates it's state every clock cycle of the ROS timer every **0.5 seconds**. On each cycle the matching function handler is called from the following handlers as can be seen in the following listing:

Listing 3: Mission loop dispatch

```
def _mission_loop(self):            # called by ROS timer every 0.5 s
    handlers = {
        PLANNING:               self._do_planning,
        NAVIGATING_TO_PICKUP:   self._do_nav_pickup,
        DETECTING_MARKER:       self._do_detect,
        CONFIRMING_PICKUP:      self._do_confirm,
        NAVIGATING_TO_DROPOFF:  self._do_nav_dropoff,
        DELIVERING:             self._do_deliver,
    }
    handlers[self.state]()          # runs only the current state's function
```

Why 2 Hz is enough. Most states either (a) immediately transition to the next state (PLANNING, CONFIRMING, DELIVERING take < 1 ms to execute), or (b) block waiting for Nav2 to finish navigating. Detection frequency of 2Hz is sufficient because the camera publishes at 8–10 Hz i.e. the marker is detected long before the next timer tick.

The `_nav_in_progress` flag. The Navigation is a long-duration operation (the robot may drive for 30–60 seconds). During this time the timer ticks again every 0.5 s. To prevent launching a *second* navigation call while the first is still running, a boolean flag

`_nav_in_progress` is set to `True` before calling `navigate_to()` and cleared to `False` after it returns. Each navigation handler checks this flag first:

Listing 4: Guard flag preventing re-entry

```
def _do_nav_pickup(self):
    if self._nav_in_progress:   # timer fired again while navigating skip
        return
    self._nav_in_progress = True
    ok = self.navigator.navigate_to(x, y, theta)   # blocks until done
    self._nav_in_progress = False
    # now transition to next state based on ok
```

The timer loop runs as normal, but the navigation logic is executed only once at a time.

Approach Point Computation

Rather than navigating directly to the zone origin (which may be occupied by the spawned object), the controller computes an *approach point* 0.4 m from the target, on the line from the current robot position to the target, optionally rotated by an `angle_offset`:

$$\mathbf{p}_{\text{approach}} = \mathbf{g} + R(\theta_{\text{offset}}) \frac{\mathbf{r} - \mathbf{g}}{|\mathbf{r} - \mathbf{g}|} d$$

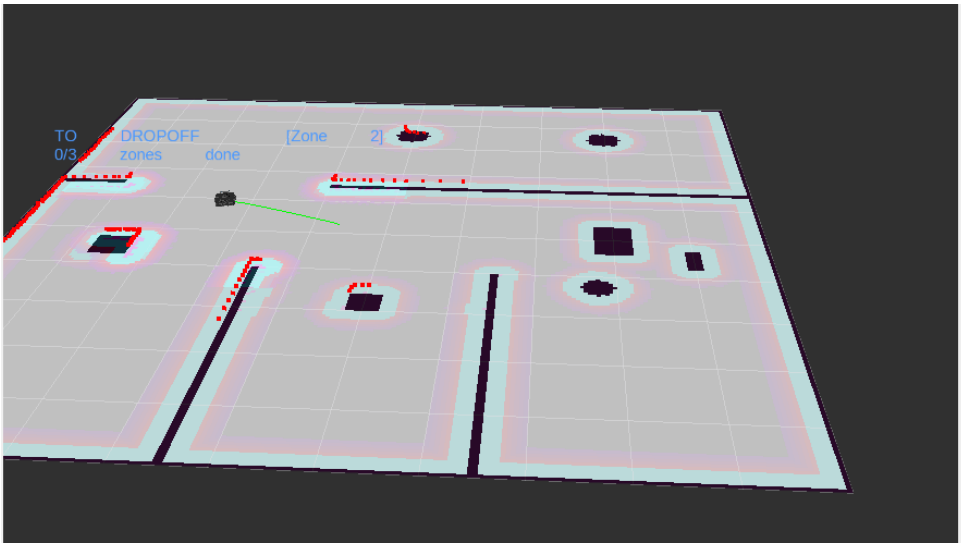
where $\mathbf{g}$ is the goal position, $\mathbf{r}$ is the robot position, $d = 0.4\,\text{m}$, and $R(\theta_{\text{offset}})$ is a 2-D rotation matrix. The heading is set so the robot faces the marker upon arrival: $\theta = \text{atan2}(g_y - p_y,\ g_x - p_x)$.

Multi-Angle Retry

If a navigation goal fails (obstacle, timeout, etc.), the controller retries from three alternative approach angles before skipping the zone:

Attempt	Offset	Direction
1	0	Direct (from robot toward goal)
2	+90	Left side of goal
3	−90	Right side of goal
4	180	Behind goal

RViz Status Overlay



A `visualization_msgs/Marker` of type `TEXT_VIEW_FACING` is published to `/mission_status_marker` on every state transition. The marker floats 0.8 m above the robot base link and displays the current state and zone progress in colour-coded text: blue for navigation, yellow for detection/confirmation, and green for completion.

`waypoint_navigator` — Nav2 Action Client

This node wraps the Nav2 `NavigateToPose` action server in a simple *blocking* API call.

Public Interface

Listing 5: WaypointNavigator public interface

```
def navigate_to(self, x, y, theta=0.0, timeout_sec=120.0) -> bool
    :
    """Send robot to (x, y) facing theta. Blocks until done.
       Returns True on success."""

def cancel_navigation(self):
    """Cancel the active goal."""
```

Execution Flow

1. Wait up to 10 s for the Nav2 action server.
2. Build a `geometry_msgs/PoseStamped` in the `map` frame, converting the yaw θ to a quaternion: $q = (0,\ 0,\ \sin(\theta/2),\ \cos(\theta/2))$.
3. Send goal asynchronously via `send_goal_async`.
4. Poll in 50 ms intervals until goal acceptance (15 s max).
5. Poll in 100 ms intervals until navigation result or timeout.
6. Return `True` iff `GoalStatus.STATUS_SUCCEEDED`.

The polling loop (rather than `spin_until_future_complete`) allows the `MultiThreadedExecutor` to continue processing other callbacks (e.g. the marker subscription) while navigation is in progress.

`marker_detector` — ArUco Visual Detection

This node subscribes to the robot's camera feed and publishes a list of detected ArUco marker IDs on every frame.

ArUco Detection Pipeline

1. Subscribe to `/camera/image_raw` (TurtleBot3 Waffle front camera).
2. Convert the ROS `sensor_msgs/Image` to an OpenCV BGR image via `CvBridge`.
3. Convert to grayscale (`cv2.COLOR_BGR2GRAY`).
4. Run the ArUco detector with dictionary `DICT_4X4_50`.
5. Publish the detected IDs as `std_msgs/Int32MultiArray` on `/aruco/marker_ids`.

Detection Parameter Tuning

The default OpenCV ArUco parameters are tuned for Gazebo's software-rendered (llvmpipe) images which lack sharp contrast gradients:

Parameter	Default	Tuned value
`minOtsuStdDev`	5.0	3.0
`adaptiveThreshWinSizeMax`	23	30
`minMarkerPerimeterRate`	0.03	0.02
`polygonalApproxAccuracyRate`	0.05	0.08

Lowering `minOtsuStdDev` allows detection in low-contrast rendered images; widening `adaptiveThreshWinSizeMax` handles uneven lighting; reducing `minMarkerPerimeterRate` detects smaller markers at distance.

`task_scheduler` — Zone Ordering

This node loads the zone configuration from `waypoints.yaml` and decides the visit order using one of two strategies.

Bonus Implementation: Distance-Based (Nearest First)

At each planning step, the Euclidean distance from the robot's current position to every unvisited zone is computed:

$$d_i = \sqrt{(x_i - x_r)^2 + (y_i - y_r)^2}$$

The zone with the minimum d_i is selected. This is a **greedy nearest-neighbour** heuristic for the Travelling Salesman-like ordering problem, providing a practical sub-optimal solution that minimises total travel distance without expensive combinatorial search.

Priority-Based: Was implemented earlier, C

Each zone in `waypoints.yaml` carries an integer `priority` field. Zones are sorted in ascending priority order (1 = highest), regardless of distance.

Listing 6: Strategy selection in get_next_task()

```
if self.strategy == 'distance':
    remaining.sort(key=self._dist)
else:
    remaining.sort(key=lambda z: z.get('priority', 99))
return remaining[0]
```

Completed zone IDs are stored in a `set` (`completed_zones`). A zone is added to the set either after successful delivery or after exhausting all four approach retries. `get_next_task()` filters the `pickup_zones` list against this set on every call.

`pickup_site_node` — Gazebo Zone Spawning

This node has two responsibilities:

1. Generate random, collision-free positions for the three pickup zones.
2. Spawn the visual objects (coloured platform + ArUco marker cube + coloured object geometry) into Gazebo at the collision-free positions.

Random Position Generation

The sampling algorithm (`_is_valid()` and `_random_positions()`) is described in full in Section 3.3 (Exclusion-Zone Sampling), including the validity conditions, the double loop, and the fallback positions. The accepted (x, y) values are injected into the scheduler's waypoint table by `main_controller._apply_random_zone_positions()` at startup.

Gazebo Spawning

Each zone spawns three SDF models via the `/spawn_entity` Gazebo service:

Model suffix	Description	z
`_platform`	0.6 m × 0.6 m coloured flat disc	0.01 m
`_marker`	0.15 m ArUco textured cube	0.095 m
`_object`	Coloured box or cylinder	0.10 m

The ArUco marker model uses Gazebo's `model://` material URI system to apply a pre-generated texture (rendered once at launch time by the launch file). Spawn calls are made asynchronously with a 10 s deadline per model. Spawned models are declared with the tag

```
<static>true</static>
```

so that they don't interfere with the costmap.

Nav2 Stack Integration

The project uses Nav2 for the complete navigation logic. The custom code issues `NavigateToPose` action goals and reads the outcome.

The Nav2 stack components active in this project:

Component	Role
`map_server`	Serves the pre-built static occupancy grid
`amcl`	Monte Carlo localisation (particle filter)
`planner_server`	A* global path planning (`NavfnPlanner`)
`controller_server`	Regulated Pure Pursuit local controller
`bt_navigator`	Behaviour tree wrapping all of the above
`local_costmap`	5 m rolling window, LIDAR obstacle layer
`global_costmap`	Full-map static + obstacle + inflation layers

Global Planner — A* (NavfnPlanner)

`NavfnPlanner` (Navigation Function Planner) is Nav2's built-in global path planner. It operates on the occupancy grid costmap and computes a collision-free path from the robot's current pose to the goal by propagating a navigation potential function across the grid cells.

With `use_astar flag as true` it computes a globally optimal path from the robot's current position to the goal on the **global costmap**.Used A* algorithm uses a Euclidean heuristic $h(n) = \|n - \text{goal}\|$ to guide search toward the goal, reducing the number of expanded nodes in open indoor environments.

Local Controller — Regulated Pure Pursuit (RPP)

DWB (Dynamic Window Approach), the Nav2 default, was found to produce "no valid trajectories" errors on curved path segments near walls because its forward-projection sampling would touch the inflated costmap. RPP was adopted as the replacement because it follows the global path via a *lookahead point* without sampling trajectories, eliminating that failure mode entirely.

RPP Parameters

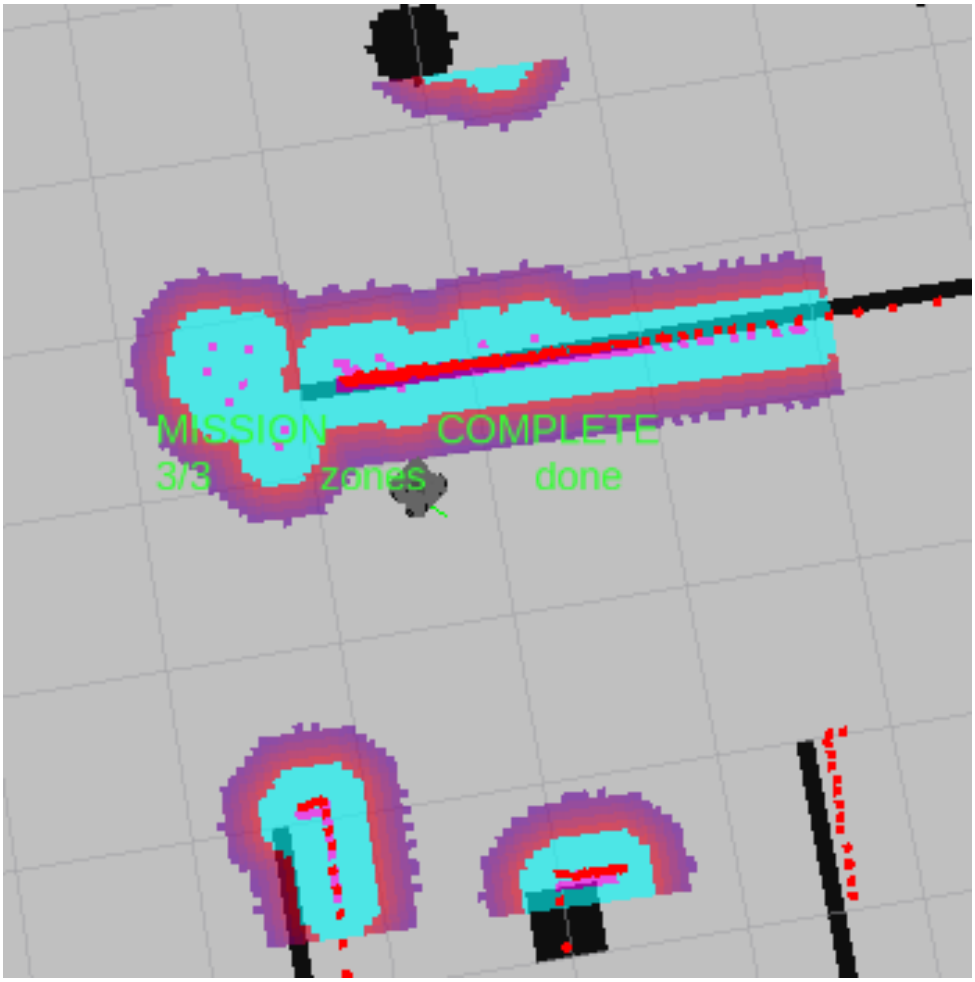
All RPP parameters are defined in `config/nav2_params.yaml` under the `FollowPath` key and loaded by Nav2's `controller_server` at launch — no changes to Python code are needed to retune controller behaviour.

Parameter	Value	Purpose
`desired_linear_vel`	0.18 m/s	Cruise speed
`lookahead_dist`	0.8 m	Base lookahead distance
`min_lookahead_dist`	0.4 m	Tight-corner fallback
`max_lookahead_dist`	1.2 m	High-speed lookahead cap
`use_velocity_scaled_lookahead_dist`	true	Scale with speed
`use_rotate_to_heading`	true	Rotate-in-place at goal
`rotate_to_heading_min_angle`	0.3 rad	Threshold for rotation
`rotate_to_heading_angular_vel`	0.8 rad/s	Rotation speed

Costmaps

A costmap is a 2-D grid that assigns a cost value to each cell in the environment, where high-cost cells represent obstacles or their immediate vicinity and low-cost cells represent free space. Nav2 maintains two separate costmaps: a *local* costmap for short-range reactive avoidance and a *global* costmap for full-map path planning.

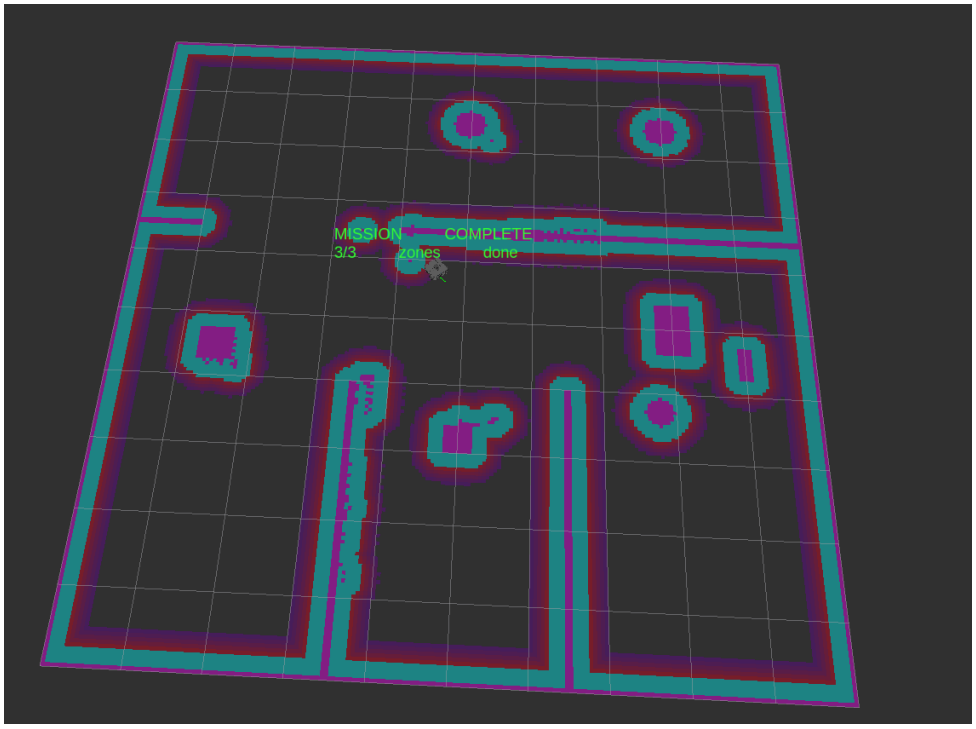
Local Costmap



The local costmap is a 5 m × 5 m window that moves with the robot, always keeping the robot at its centre. It contains a single `ObstacleLayer`, which marks cells as occupied whenever the LIDAR scan (`/scan`) reports a return within that area. Because the

environment is a flat indoor floor with no elevated obstacles, a 3-D voxel grid is unnecessary; the 2-D layer is both sufficient and computationally lighter. An `InflationLayer` then expands each occupied cell outward by 0.45 m with an exponential cost decay (factor 4.0), creating a buffer zone that the RPP controller treats as high-cost. This buffer ensures the robot maintains a safe clearance from walls and obstacles even when the planned path passes close to them.

Global Costmap



The global costmap covers the entire 10 m × 10 m environment and is used by the A* planner to compute an initial collision-free path before each navigation goal. It is composed of three layers applied in order:

1. `StaticLayer` — incorporates the pre-built PGM occupancy grid, marking all known walls and fixed obstacles.
2. `ObstacleLayer` — overlays live LIDAR readings, allowing the planner to account for any dynamic or previously unmapped obstacles.
3. `InflationLayer` — applies the same 0.45 m inflation radius as the local costmap, so planned paths maintain clearance from all obstacle boundaries.

Localisation — AMCL

Adaptive Monte Carlo Localisation (AMCL) is a probabilistic algorithm that estimates the robot's position and orientation within a known map. It maintains a *particle filter*: a set of weighted hypotheses, each representing a possible robot pose. As the robot moves, the particle filter is updated using the odometry model (to predict how the robot moved) and the LIDAR model (to score each hypothesis against the actual sensor reading). Over time, particles converge around the true pose.

The robot is initialised at the map origin $(0, 0)$ with a known starting pose, so the

particle filter begins concentrated rather than uniformly distributed across the map. Three parameters were tuned for stable operation in the software-rendered Gazebo environment:

- **Motion noise** $\alpha_{1-5} = 0.2$: These coefficients scale the uncertainty added to the odometry model on each step. A low value reflects the fact that Gazebo's simulated odometry is relatively clean and consistent.
- **Sensor budget**: `max_beams = 60` limits the number of LIDAR rays used per update step, and `max_particles = 2000` caps the particle count. Both values balance localisation accuracy against the CPU cost of running AMCL alongside software-rendered Gazebo.
- **Transform tolerance** = 3.0 s: AMCL publishes a `map` → `odom` transform at each update. A high tolerance prevents the navigation stack from rejecting the transform as stale during the brief slowdowns caused by Gazebo's variable real-time factor under software rendering.

Goal Tolerances

Nav2 declares a navigation goal *reached* only when both of the following conditions are simultaneously satisfied:

Parameter	Value	Meaning
`xy_goal_tolerance`	0.20 m	The robot must be within 20 cm of the goal position
`yaw_goal_tolerance`	0.15 rad (≈ 8.6)	The robot must be facing within 8.6° of the goal heading

The detection phase requires the ArUco marker to appear within the camera's horizontal field of view (≈60°). If the robot arrives at the correct position but its heading is slightly off-axis, the marker may fall outside the camera's view and go undetected. Setting `yaw_goal_tolerance` to 0.15 rad ensured that the robot is facing the marker before the detection phase begins.

This has been implemented by setting `rotate_to_heading_min_angle = 0.3 rad` i.e. whenever the robot's heading error at the goal exceeds 0.3 rad (≈ 17), RPP performs a rotate-in-place manoeuvre before declaring arrival, correcting the heading before the timer transitions to the `DETECTING_MARKER` state.

ROS2 Communication Graph

Topics

Topic	Message Type	Description
`/camera/image_raw`	`sensor_msgs/Image`	Raw camera frames from Waffle
`/scan`	`sensor_msgs/LaserScan`	360° LIDAR scan
`/odom`	`nav_msgs/Odometry`	Wheel odometry from Gazebo
`/aruco/marker_ids`	`std_msgs/Int32MultiArray`	Detected ArUco IDs each frame
`/mission_state`	`std_msgs/String`	Current FSM state
`/mission_status_marker`	`visualization_msgs/Marker`	RViz text overlay
`/map`	`nav_msgs/OccupancyGrid`	Static map from `map_server`
`/tf`, `/tf_static`	`tf2_msgs/TFMessage`	Robot transform tree
`/cmd_vel`	`geometry_msgs/Twist`	Velocity commands to Gazebo

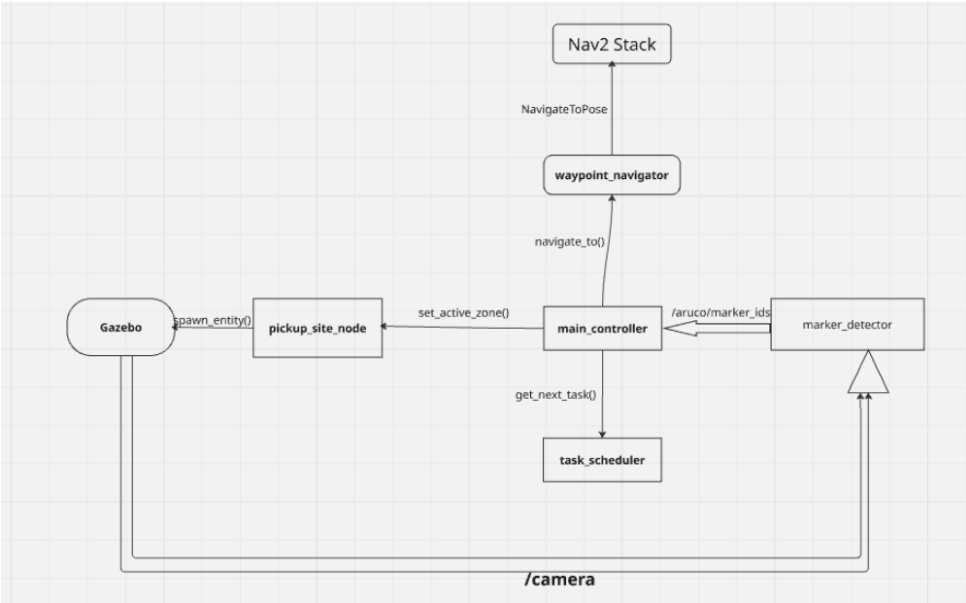
Services

Service	Type	Description
`/start_mission`	`std_srvs/Trigger`	Start or restart the mission
`/stop_mission`	`std_srvs/Trigger`	Abort current mission
`/spawn_entity`	`gazebo_msgs/SpawnEntity`	Spawn SDF models into Gazebo

Actions

Action	Type	Description
`/navigate_to_pose`	`nav2_msgs/NavigateToPose`	Send robot to a goal pose (blocks)

Node Communication Diagram



Fat and triangular arrows represent topic subscriptions; lean arrows represent direct method calls (same-process sub-nodes) or service/action calls.

Task Scheduling

We initially have $n = 3$ pickup zones at positions $\{P_1, P_2, P_3\}$ and robot position R, the task scheduling problem is to find an ordering $\sigma : \{1, 2, 3\} \to \{1, 2, 3\}$ that either:

- Minimises total travel distance $\sum_{k=0}^{n-1} d(R_k, P_{\sigma(k+1)})$ (distance mode), or
- Respects pre-assigned priority values π_i (priority mode).

With only three zones, the full optimal ordering could in principle be found by enumeration ($3! = 6$ permutations). However, the greedy nearest-neighbour approach is used instead because it generalises trivially to n zones and is called *at each planning step* (after delivery), so it naturally adapts to the robot's new position.

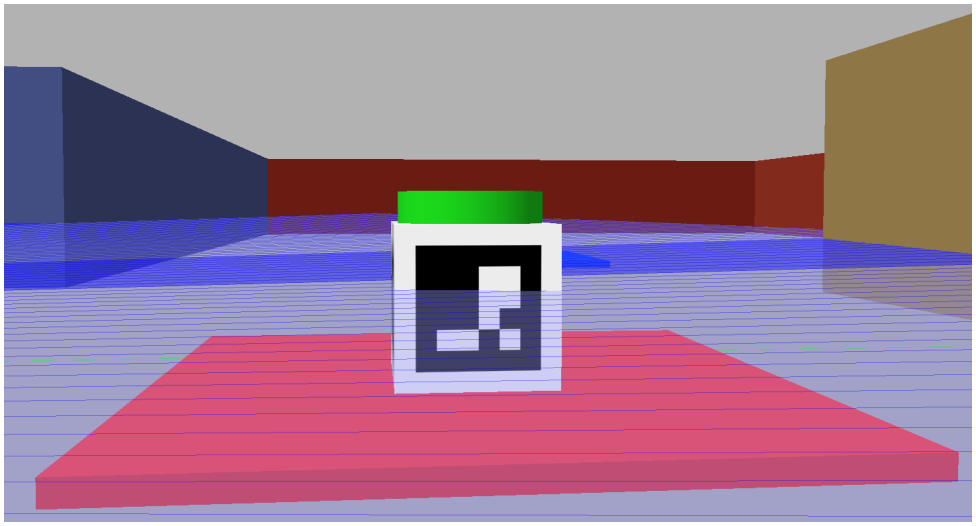
Scheduling Strategies

Distance (nearest-first): At each step, sort remaining zones by Euclidean distance from robot and visit the closest.

Priority(not implemented in the latest version): Sort remaining zones by the integer `priority` field in `waypoints.yaml` (lower = higher priority) and visit them in order, irrespective of distance. Useful when some objects are time-sensitive.

The strategy is configurable via a ROS2 parameter (`scheduling_strategy`) at launch time.

ArUco Marker Detection and Object Identity Confirmation



ArUco markers are binary square fiducial markers from a predefined dictionary. Each marker encodes a unique integer ID in its black-and-white grid pattern. This project uses `DICT_4X4_50`: a 4×4 interior grid, supporting up to 50 unique IDs. The three pickup zones carry markers with IDs 0, 1, and 2 respectively.

Zone–Marker Mapping

Zone	Marker ID	Platform colour	Object
Zone 1	0	Green	Red Box (1.2 kg)
Zone 2	1	Blue	Blue Cube (0.8 kg)
Zone 3	2	Red	Green Cylinder (1.5 kg)

Each marker is a 0.15 m cube mounted at 0.095 m height (roughly the TurtleBot3 Waffle camera centre at ≈0.10 m). The Gazebo model uses Ogre material scripts to texture all six faces of the cube with the correct ArUco pattern.

Detection-Confirmation Protocol

The state machine enforces a two-stage confirmation:

1. **DETECTING_MARKER**: The controller checks `detected_markers` (updated by `/aruco/marker_ids` subscription at every camera frame). Detection is successful only when the *expected* marker ID (defined per zone in `waypoints.yaml`) appears in the list. A **15-second timeout** exists as a fallback in case the marker is blocked from the camera view or the camera angle is imperfect; the timeout triggers a warning log and proceeds, but does not claim a detection.
2. **CONFIRMING_PICKUP**: Once the expected marker has been seen, the controller logs the confirmed object name and transitions to dropoff. This stage represents the logical "pickup" action.

This design ensures the robot **never claims a detection without the camera actually detecting the correct marker ID**. Only a genuine match between expected and detected IDs advances the mission.

Heading Precision for Detection

A key insight is that ArUco detection requires the marker to be in the camera's field of view (≈60° horizontal). Nav2's `yaw_goal_tolerance` was tightened to 0.15 rad (≈8.6°) and `rotate_to_heading_min_angle` to 0.3 rad (≈17°), ensuring the robot faces the marker upon arrival. The approach distance was also reduced to 0.4 m to increase marker angular size in the image.

Launch System

Launch File Structure

The single launch file `full_nav2.launch.py` starts all components in sequence to ensure the environment is ready before the controller begins:

1. **Map server**: serves the pre-built PGM map.
2. **Gazebo**: loads the world file with walls, partitions, and obstacles.
3. **Robot state publisher**: broadcasts URDF-based TF tree for the Waffle.
4. **Nav2 bringup**: AMCL, planners, controllers, and costmaps.
5. **RViz2**: pre-configured view showing the map, robot, and camera feed.
6. **Marker detector node**: starts ArUco detection from the camera.
7. **Main controller node**: starts the mission FSM and waits for the trigger.

Execution and Process Management

To run the package, the following commands are executed in separate terminals.

Terminal 1: Environment Setup

```
# Force-close background processes to ensure a clean start
pkill -9 -f gzserver; pkill -9 -f gzclient; pkill -9 -f rviz2;
pkill -9 -f component_container; pkill -9 -f ros2; sleep 2

cd /workspaces/ros2_workspace
source /opt/ros/humble/setup.bash
export TURTLEBOT3_MODEL=waffle
colcon build --symlink-install
source install/setup.bash
ros2 launch object_fetcher full_nav2.launch.py gui:=true rviz:=
    true
```

Terminal 2: Mission Start

```
source install/setup.bash
ros2 service call /start_mission std_srvs/srv/Trigger "{}"
```

The `pkill` commands are necessary because Gazebo and RViz often leave old processes running in the background after a terminal is closed. If these are not cleared, they cause conflicts, preventing the new simulation from loading correctly.

Starting a Mission

After launch, the mission begins with a single service call:

```
ros2 service call /start_mission std_srvs/srv/Trigger "{}"
```

The robot then executes the full pickup-detect-dropoff cycle for all three zones autonomously.

System Integration and Design Decisions

Single-Process Multi-Node Architecture

A key architectural decision is that all five nodes run in a **single OS process** under a `MultiThreadedExecutor` with four threads. This approach has been used to implement a modular architecture.

RPP over DWB

The Dynamic Window Approach (DWB) controller was initially used but rejected because its trajectory sampling would report "no valid trajectories" on curved path segments near inflated walls — causing the robot to abort navigation even when a path clearly existed. RPP solves this by *following the global path directly* via a lookahead point, rather than exhaustively sampling velocity commands.

2-D vs. 3-D Costmap

The local costmap uses a flat `ObstacleLayer` (2-D) rather than the default `VoxelLayer` (3-D). Because the robot operates on a flat floor and all obstacles are ground-level, the 3-D voxel grid is unnecessary.

Randomised Zone Positions

Pickup zones are randomised at every launch, not fixed in YAML. The YAML file contains only structural zone metadata (IDs, priorities, marker IDs); the spatial coordinates are generated at runtime by `pickup_site_node` and passed into the scheduler's waypoint list before the first planning step. This tests that the robot's navigation strategy is genuinely general, not memorised.

Results and Testing

Observed Behaviour

The system has been verified to complete the full mission (3/3 zones) in multiple test runs with varying random zone positions. Key observations:

- **Marker detection**: The camera successfully detects the correct ArUco marker when the robot faces within ∼20° of the marker face. If the robot faces with higher approach offset angle, the detection does fail.
- **Navigation**: RPP significantly improved the robot's ability to handle tight curves as opposed to our older DWB implementation. DWB used to overestimate the cost of going on such paths, RPP effectively manages this by regulating linear velocity based on the curvature of the path
- **Retry logic**: The four-angle retry mechanism has been observed to recover from cases where a spawned zone object partially blocks the direct approach point.
- **Scheduling**: Distance-based scheduling visibly selects closer zones first, reducing total mission travel time compared to a fixed order.

Failure Modes and Mitigations

Failure	Cause	Mitigation
Marker not detected	Robot heading slightly off, marker outside FoV	Tighter yaw tolerance; 0.4 m approach distance
Nav goal rejected	Approach point inside obstacle inflation	Retry with rotated approach angle
Gazebo spawn timeout	Headless Gazebo temporarily busy	10 s deadline with warning; mission continues

Conclusion

This project demonstrates a complete autonomous way-point based navigation system pipeline implemented in ROS2 using a modular, node-based architecture. The system successfully:

- Navigates to three randomised pickup zones using Nav2 (A* + RPP).
- Detects ArUco markers at each zone to confirm object identity.
- Returns objects to a fixed drop-off zone.

- Selects zone visit order by nearest-first distance or by priority.
- Recovers from navigation failures via multi-angle retry.

Every state transition is logged on terminal for manual verification of states, every detection is camera-verified and also logged if it succeeds, and every failure has an explicit recovery path.

Future work includes: SLAM-based mapping (removing the pre-built map requirement), and replacing the simulated pickup confirmation with a physical contact sensor or gripper.